

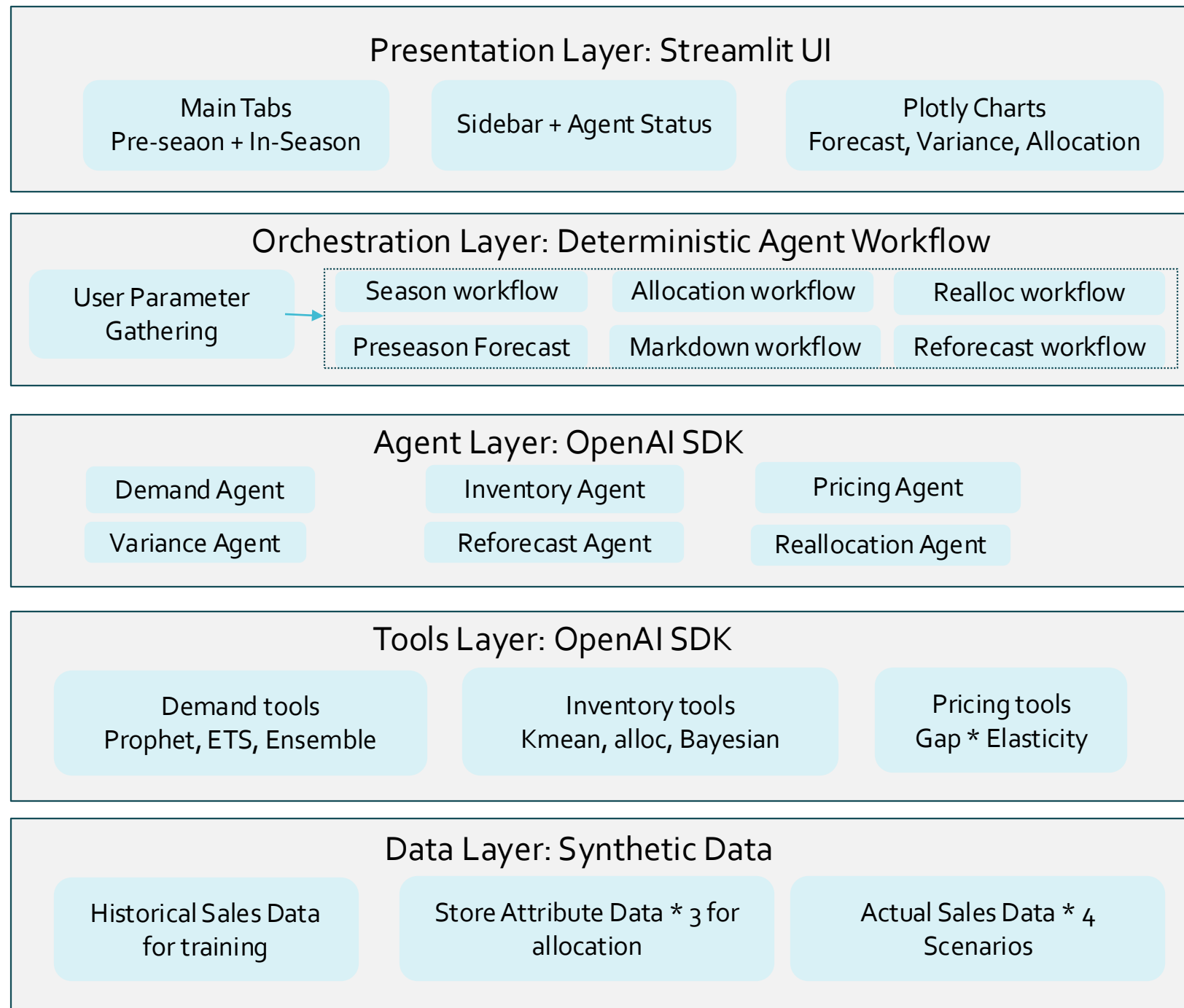
Multiagent Retail Independent Study

Boyang Wan, Henry Tang, JaeYoon Lee, Jintao Li, and Yina Liang Li

BUSA 611-613

Dec 05, 2025

Architecture



Agents Overview

Agent	Role	Key Decision	Output Schema
Demand Agent	Forecast total demand	Safety stock %, model selection	ForecastResult
Inventory Agent	Allocate to stores	DC holdback %, cluster weights	AllocationResult
Pricing Agent	Calculate markdowns	Markdown %, timing	MarkdownResult
Variance Agent	Analyze forecast accuracy	Should we reforecast?	VarianceAnalysis
Reforecast Agent	Update forecast with actuals	Posterior estimates	ReforecastResult
Reallocation Agent	Strategic replenishment	Transfer strategy	ReallocationAnalysis

Agentic Approach

- Agent Communications (input & output validation)
 - Pydantic BaseModel to validate the output and input
 - OpenAI Agent SDK Guardrails to validate agent output
- Agent Status:
 - RunHooks for real-time UI
- Tool using:
 - @function_tool
- Orchestration pattern:
 - Deterministic workflow, no handoff, no Agent-as-Tool
- Context:
 - RunContextWrapper ensures tools access ForecastingContext without LLM seeing raw data

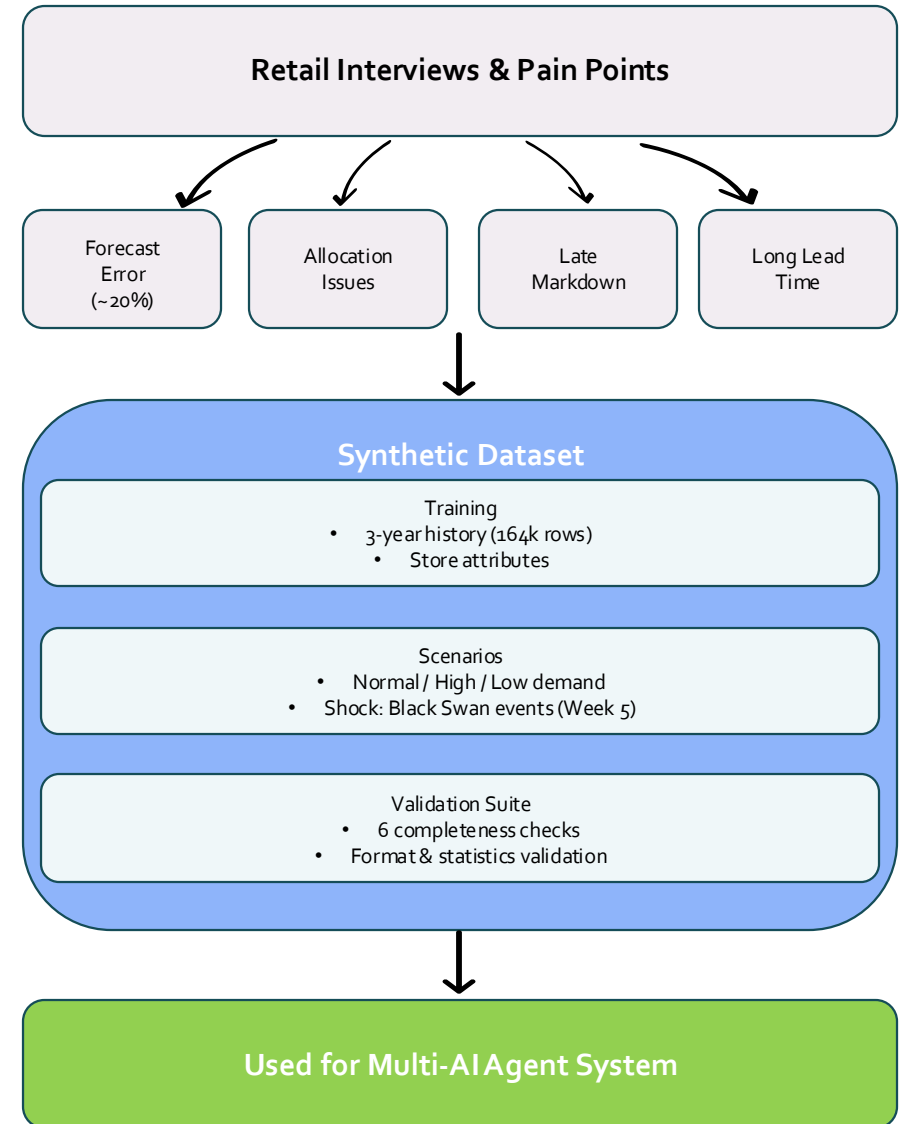
Tech Stack

Layer	Technology	Purpose
Frontend	Streamlit 1.28+	Interactive web UI
Visualization	Plotly 5.18+	Charts and graphs
Agent Framework	OpenAI Agents SDK 0.2+	Agent orchestration
LLM	GPT-4o-mini	Reasoning and generation
Forecasting	Prophet 1.1+, statsmodels	Time series models
ML	scikit-learn 1.3+	K-means clustering
Validation	Pydantic 2.0+	Schema enforcement
Data	Pandas 2.0+, NumPy	Data manipulation

Data Generation

Why and how we generated the data

- **Pain Point 1:** Inaccurate forecasts
 - Retailers reported ~20% error, especially for new seasons and launches
 - Our data: 3 years of history + noisy test scenarios with 12-18% MAPE
- **Pain Point 2:** Store-specific allocation failures
 - Teams spend hours "firefighting" stock imbalances across stores
 - Our data: 50 stores with correlated attributes
- **Pain Point 3:** Late markdowns & long lead times
 - 3 day data lag and 3-6 month manufacturing lead times make it hard to react in time
 - Our data: Weekly actuals for 12 weeks across three scenarios (normal/high/low demand) plus certain events, so the system can test variance monitoring, re-forecasting, and markdown timing



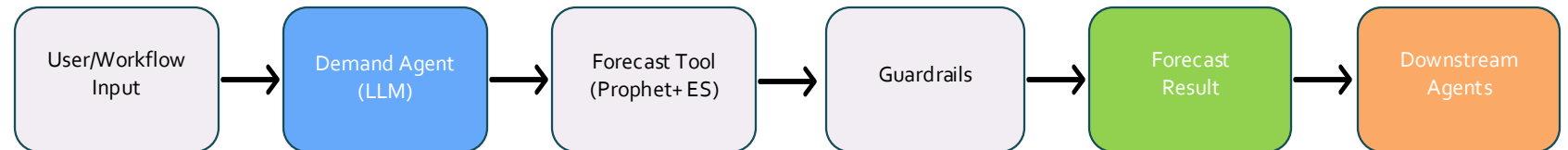
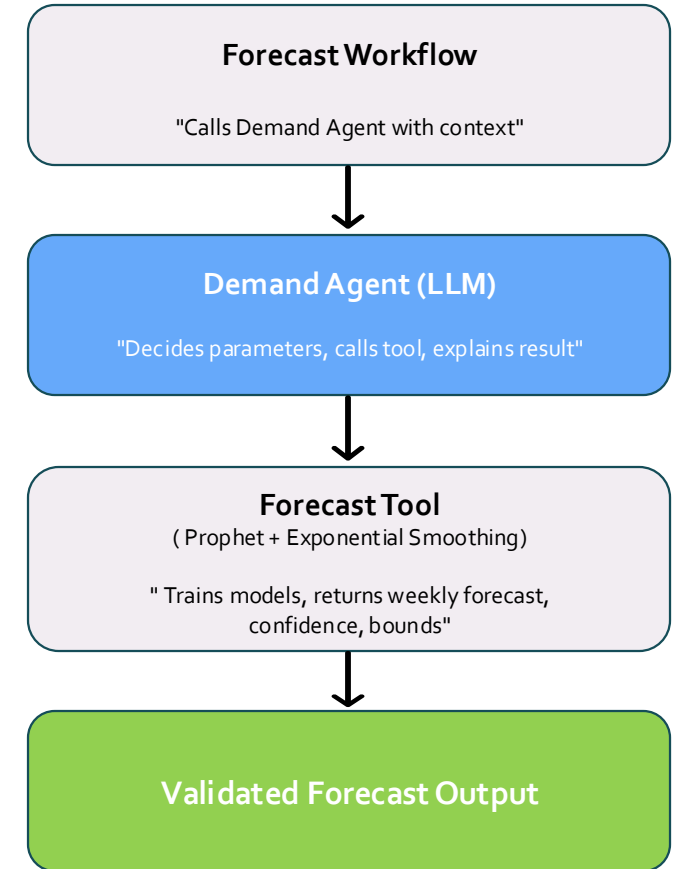
Demand Agent – Legacy vs Today Demand Model

LEGACY DEMAND MODEL	TODAY'S DEMAND MODEL
Architecture • Two-model ensemble: Prophet for trend + seasonality, ARIMA for autoregressive structure • Fixed 60/40 weighting (Prophet/ARIMA) Behavior: • Ensemble weights are static, not-data-driven • More fragile under trend breaks, shocks, and non-linear seasonality • Time consuming Performance: • ARIMA dragged down the overall performance • MAPE 10% • $R^2 \approx 0.6$	Architecture • Two-model ensemble: Prophet + Holt Winters Exponential Smoothing (ETS) • Both models are wrapped in a unified pipeline Behavior: • For each series, Prophet and ETS are trained on history and scored on a held out validation window • Ensemble weights are inverse error based • Models are then retrained on full history and combined using learned weights Performance: • Delivered roughly 19% relative MAPE improvement • R^2 improved by 18 % • Much faster

Demand Agent – Architecture

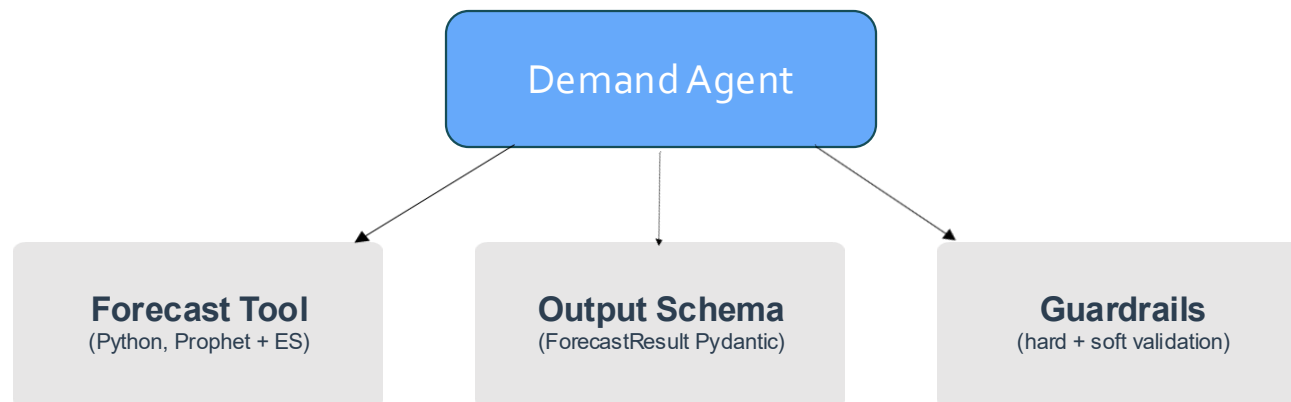
How the Demand Agent is Designed

- Three-layer design
 - **Workflow layer** – Deterministic orchestration; decides when to run the agent and passes context (category, horizon, data loader)
 - **Agent layer** – LLM reasoning; decides how to solve the forecasting problem and explains the result in business language
 - **Tool layer** – Deterministic Python tools that run Prophet + Exponential Smoothing and return raw forecast numbers
- Hybrid control philosophy
 - Deterministic layers are fully testable and reproducible
 - Agent layer adds business context, risk commentary, and safety-stock recommendations without touching raw data



Only the tool touches raw data; agent only sees sanitized tool outputs

Demand Agent – Core Components



Agent Definition



- **Name:** "Demand Forecasting Agent"
- **Role:** Combine Prophet (seasonality) and Exponential Smoothing (trend) via validation-based ensemble
- **Rule:** Must state total demand, weekly average, confidence level etc

Forecasting Tool



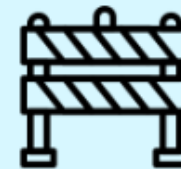
- Pulls historical sales from `ForecastingContext.data_loader`
- Cleans and aggregated to weekly data, validates ≥ 26 weeks history
- Trains a validation-based ensemble of Prophet + Exponential Smoothing and computes confidence & safety stock

Structured output



- Returns a typed object with: total demand, forecast by week, confidence, safety stock, model used
- Optional bounds and diagnostics: weekly average, data quality, lower & upper bound
- Required explanation string

Guardrails

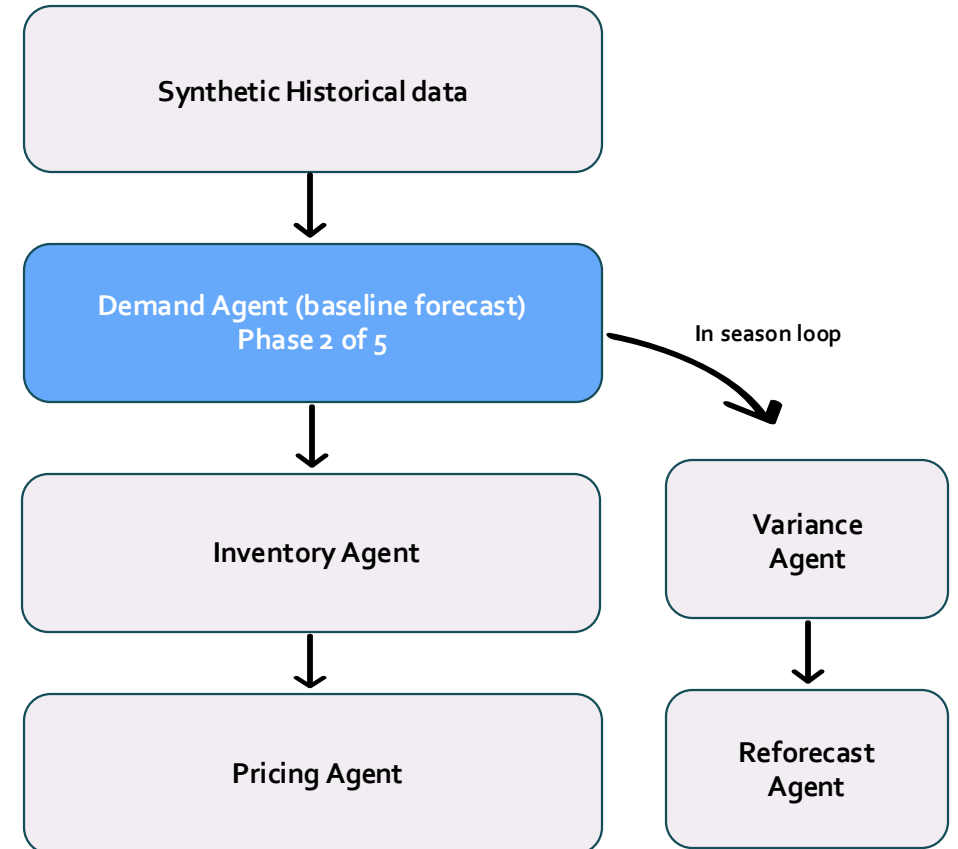


- **Hard guardrails:** Unit conservation, valid ranges for confidence and safety stock (10-50%), no negative demand, non-empty weekly list
- **Soft guardrails:** Reasonableness checks on huge totals, extreme weekly variance, safety stock vs confidence mismatch (warnings)

Demand Agent – Workflow Integration

Where the Demand Agent Fits

- **Upstream dependencies**
 - Trained on 3 years of historical data and store attributes
 - Testing scenarios feed weekly actuals back into the workflow for variance checks
- **Downstream connections**
 - Inventory Agent uses ForecastResult.total_demand and forecast_by_week to decide manufacturing quantities, DC holdback, and store cluster splits
 - Pricing Agent uses the same forecast plus confidence & data quality flags to decide pricing aggressiveness and markdown triggers
- **Feedback & Reforecast**
 - Weekly actuals from scenarios are compared to the Demand Agent's forecast
 - When variance > 20%, the workflow run reforecast to update the forecast



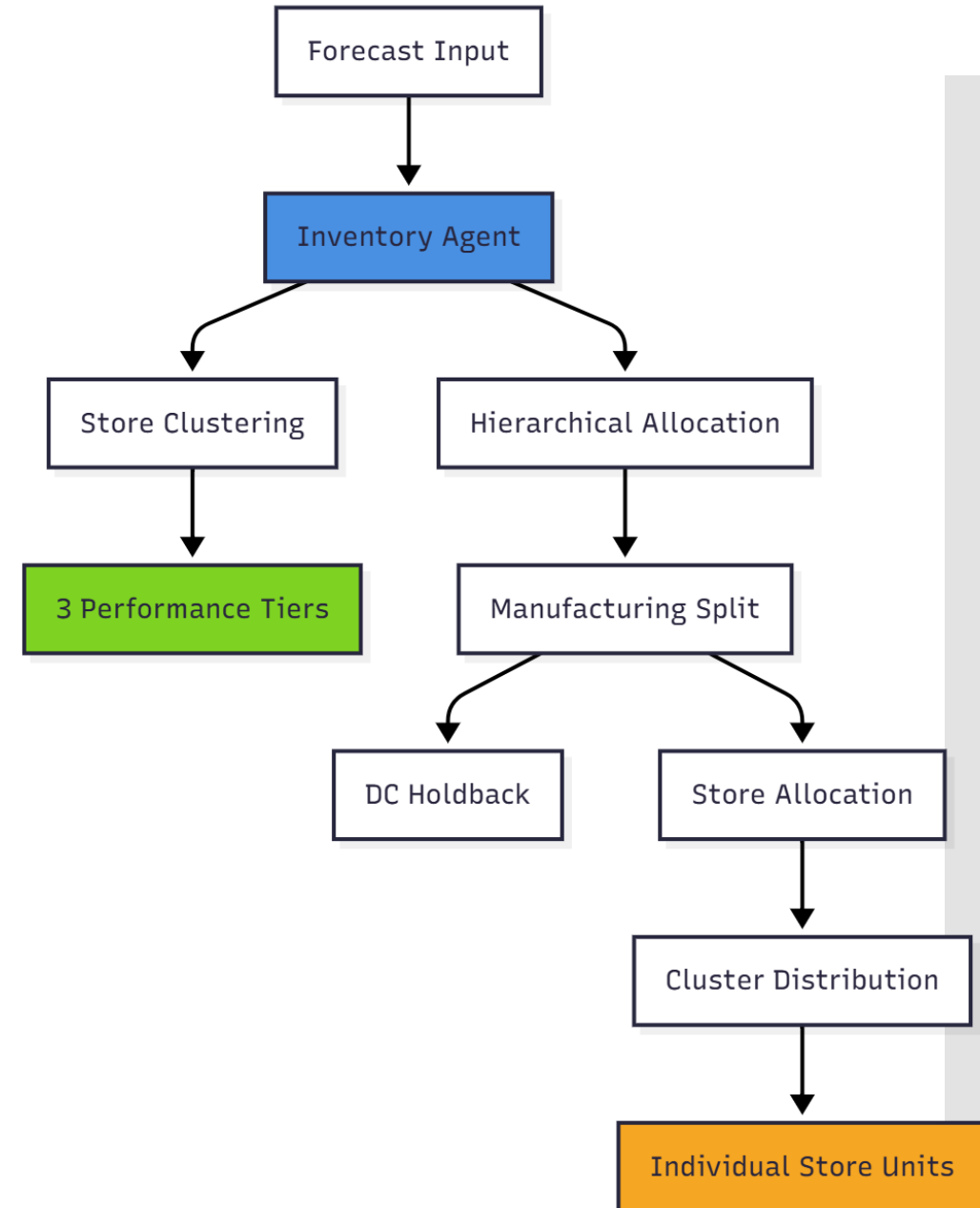
Inventory Agent: Overview

What is the Inventory Agent?

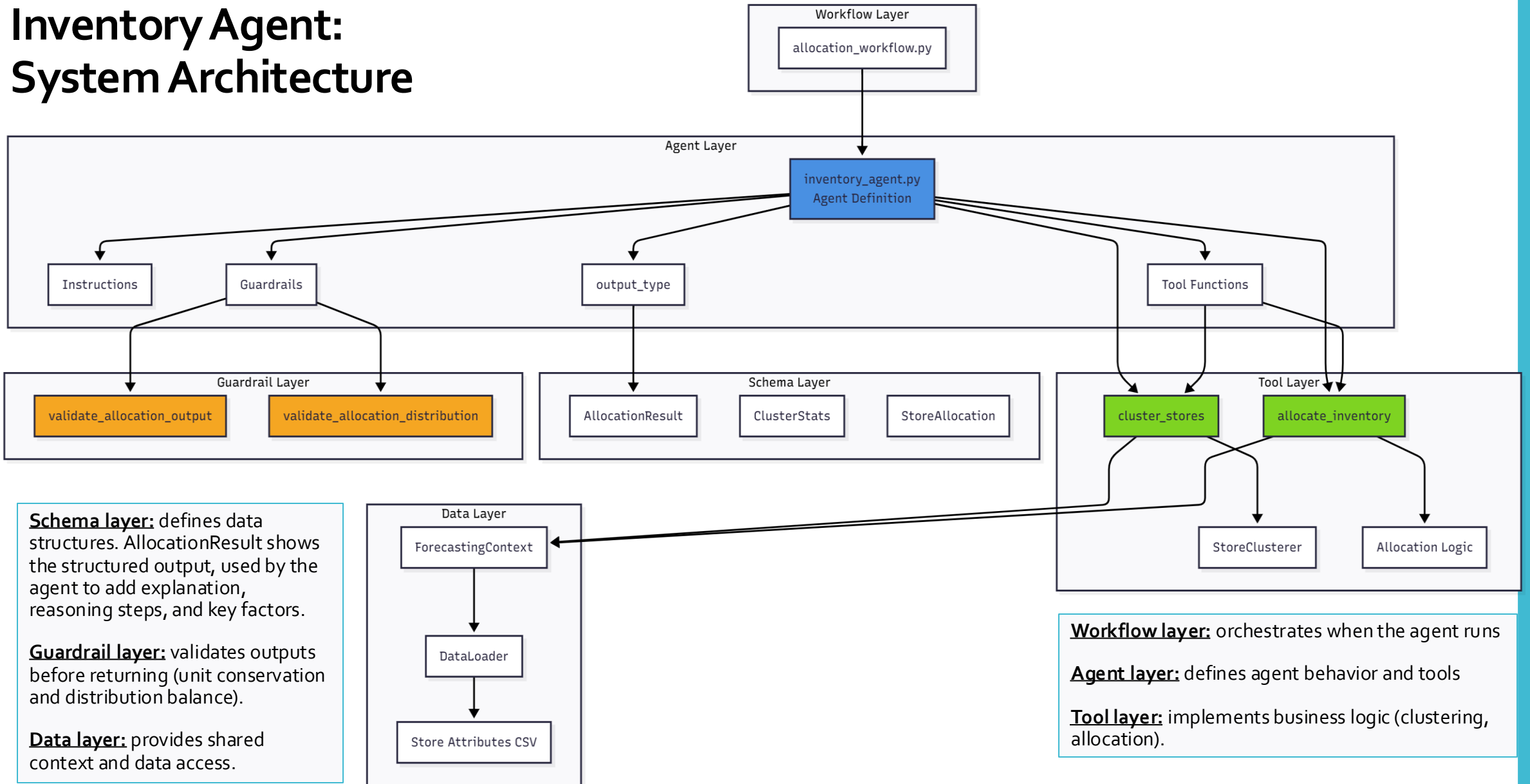
- Decides how to distribute inventory across the retail stores.
- Uses machine learning to group stores together.
- Manufacturing calculation
- Allocates inventory based on each store's performance and characteristics.
- Make sure high-performing stores get more inventory while ensuring every store has enough to operate.

Changes to the Mock Dataset:

- Rebalanced mock data (store attributes) for realistic clustering (45.7% / 41.2% / 13.1%) to avoid skewness.
 - Drives clustering and allocation.
- Added two additional realistic test datasets with different distribution patterns.
 - Tested different demand patterns, forecast accuracy, and agent behavior.
 - Proves the system is not hardcoded.



Inventory Agent: System Architecture



Inventory Agent: Clustering Process

How it Works?

Not all stores are equal.

Weighted K-means clustering to group stores into 3 tiers based on their characteristics.

Encodes categorical features and normalizes with StandardScaler.

Applies feature weights: sales represents 70% and others together 30%.

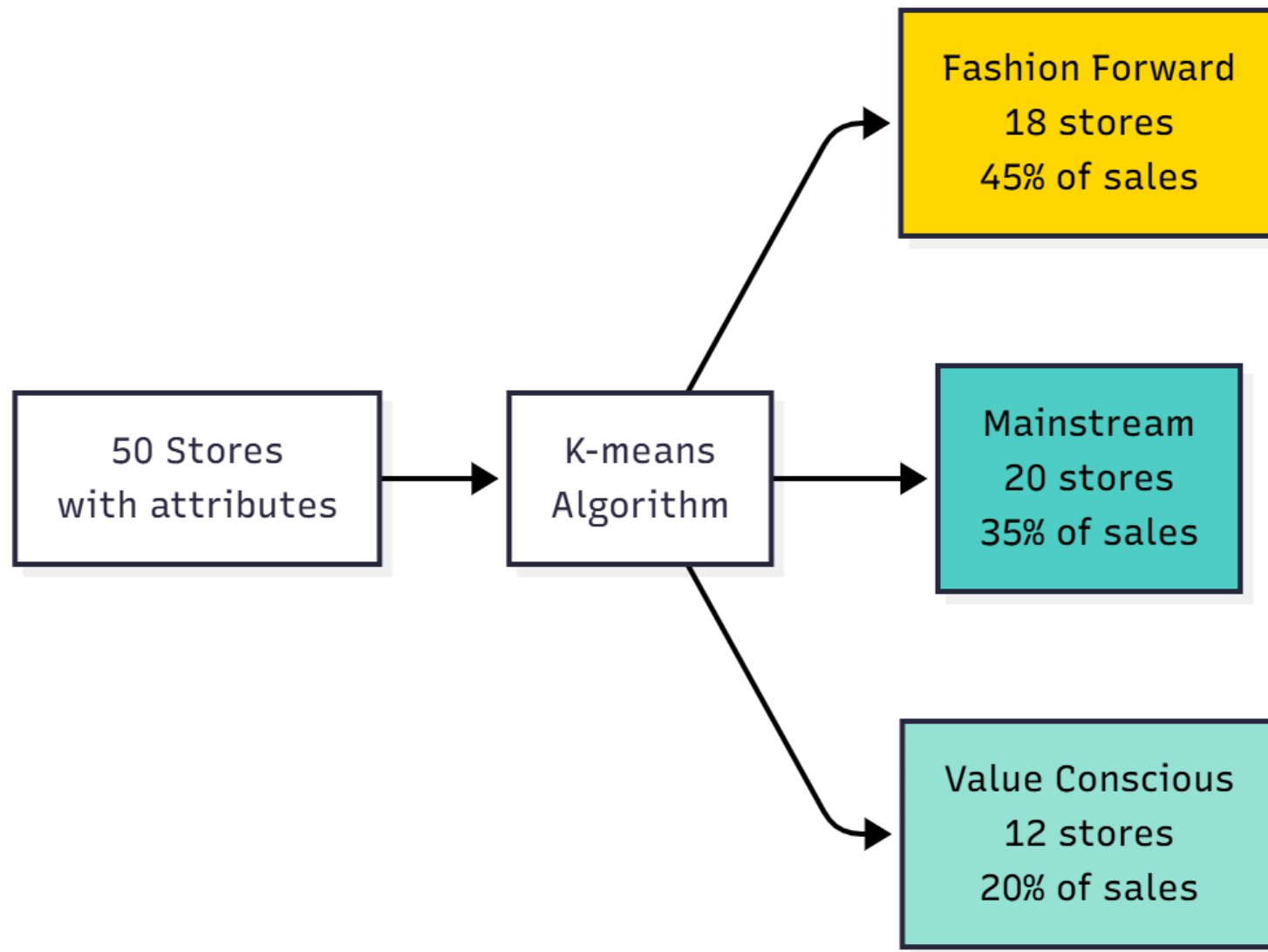
Calculates Silhouette score

Assign cluster labels

Calculates cluster statistics.

Cluster output represents 3 groups: high sales, average performance, and budget-focused.

Output percentages represent how much inventory each group gets.



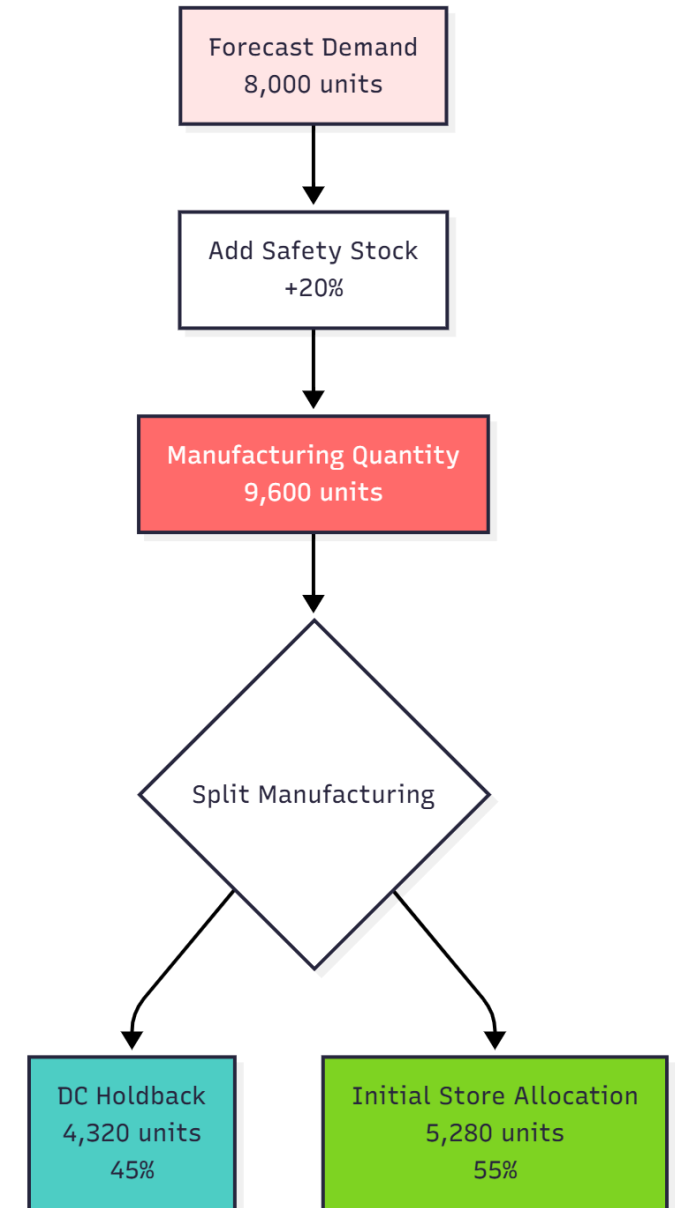
Inventory Agent: Manufacturing Calculation

How it Works?

- Base demand from forecast
- Safety stock = demand * safety stock
- Manufacturing = base + safety
- Holdback = manufacturing * DC
- Initial = manufacturing * stores split%

Why DC Holdback?

- Not sending all inventory to stores at once.
- Keep reserves at DC.
- Replenish stores weekly as they sell.
- Flexibility to respond to actual sales performance.



Inventory Agent: Allocation to Stores

How it Works? (Three-layer Hierarchy)

Layer 1: Manufacturing split between DC and store

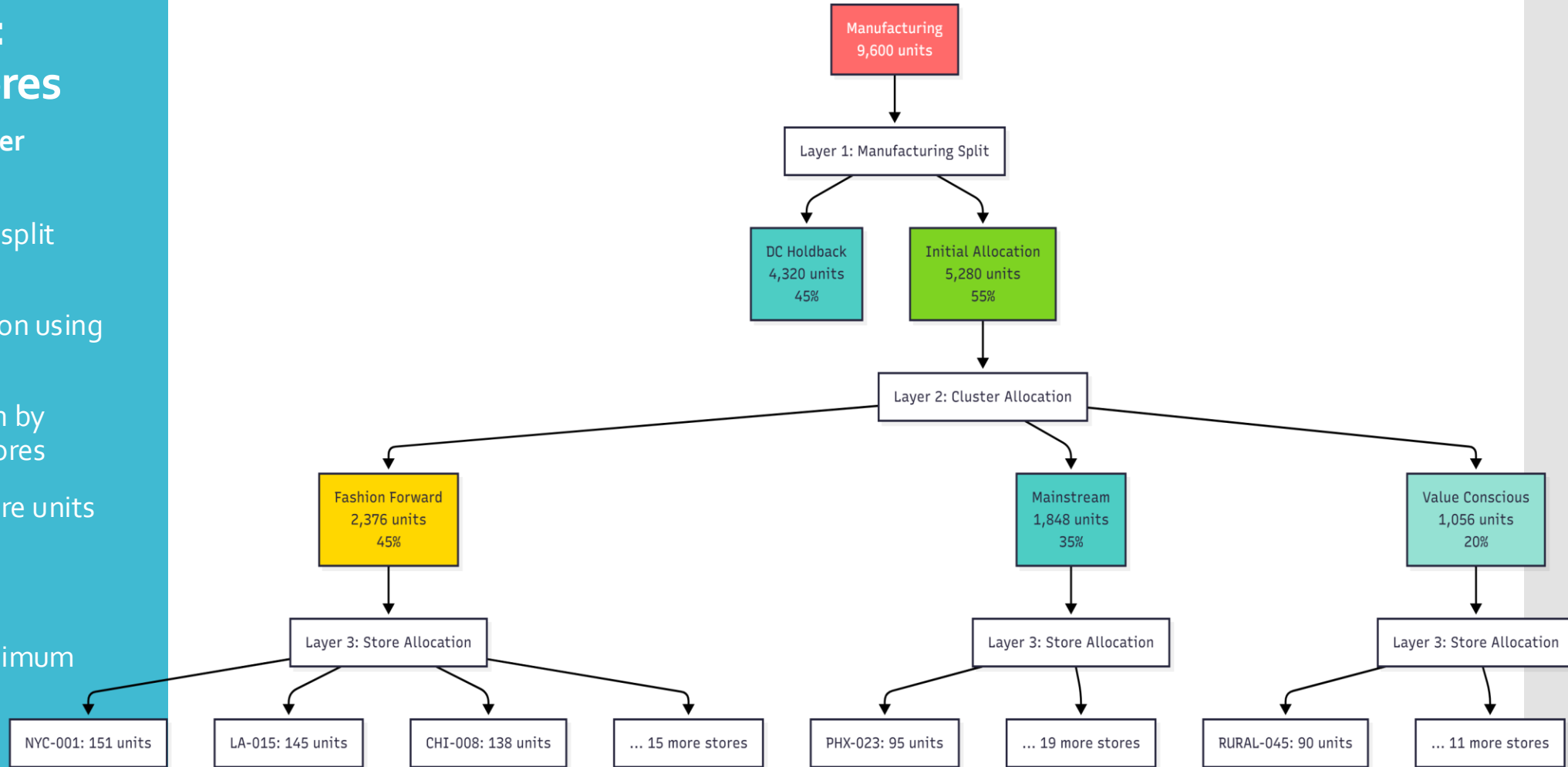
Layer 2: Cluster allocation using the cluster percentages

Layer 3: Store allocation by calculating individual stores

High performers get more units

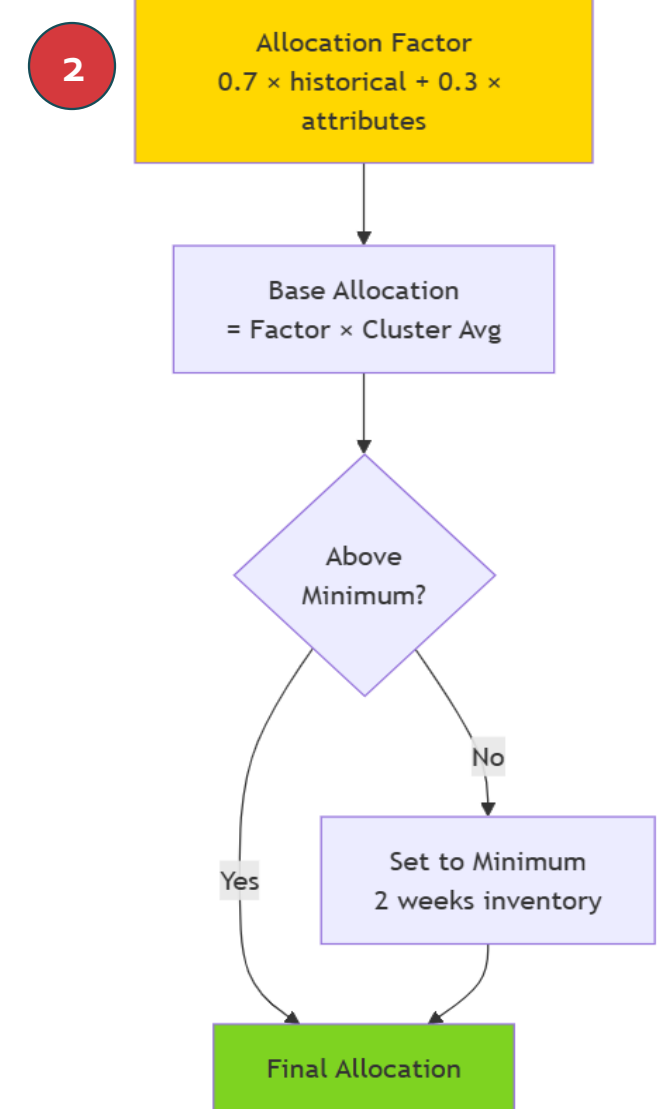
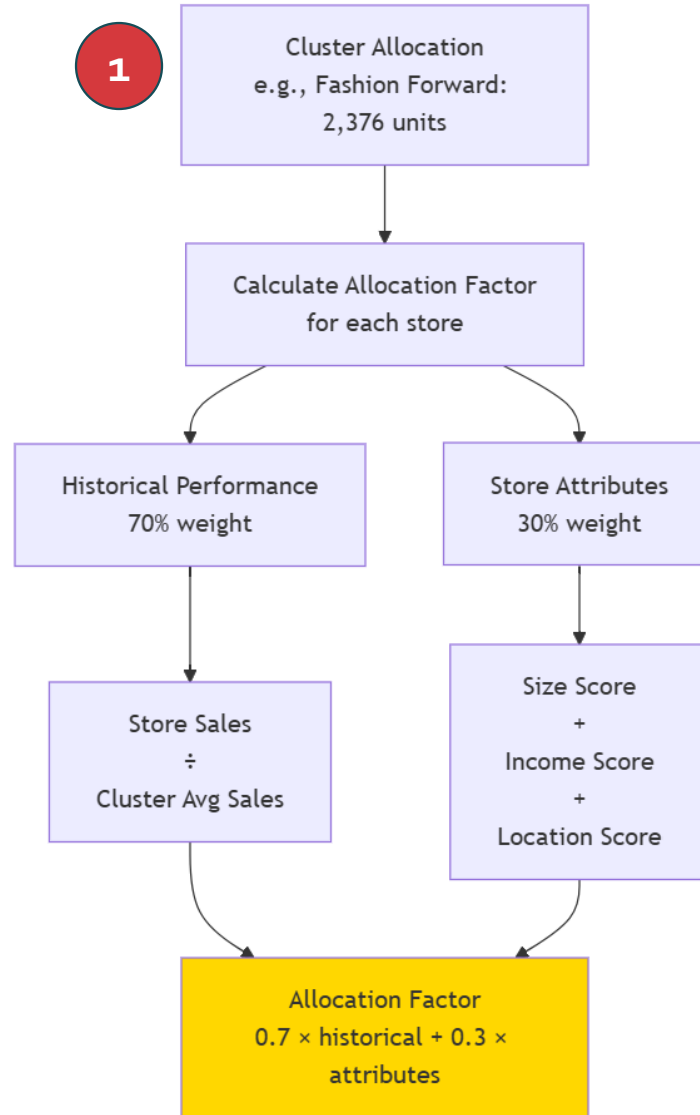
Average performers get appropriate units

Low performers get minimum units

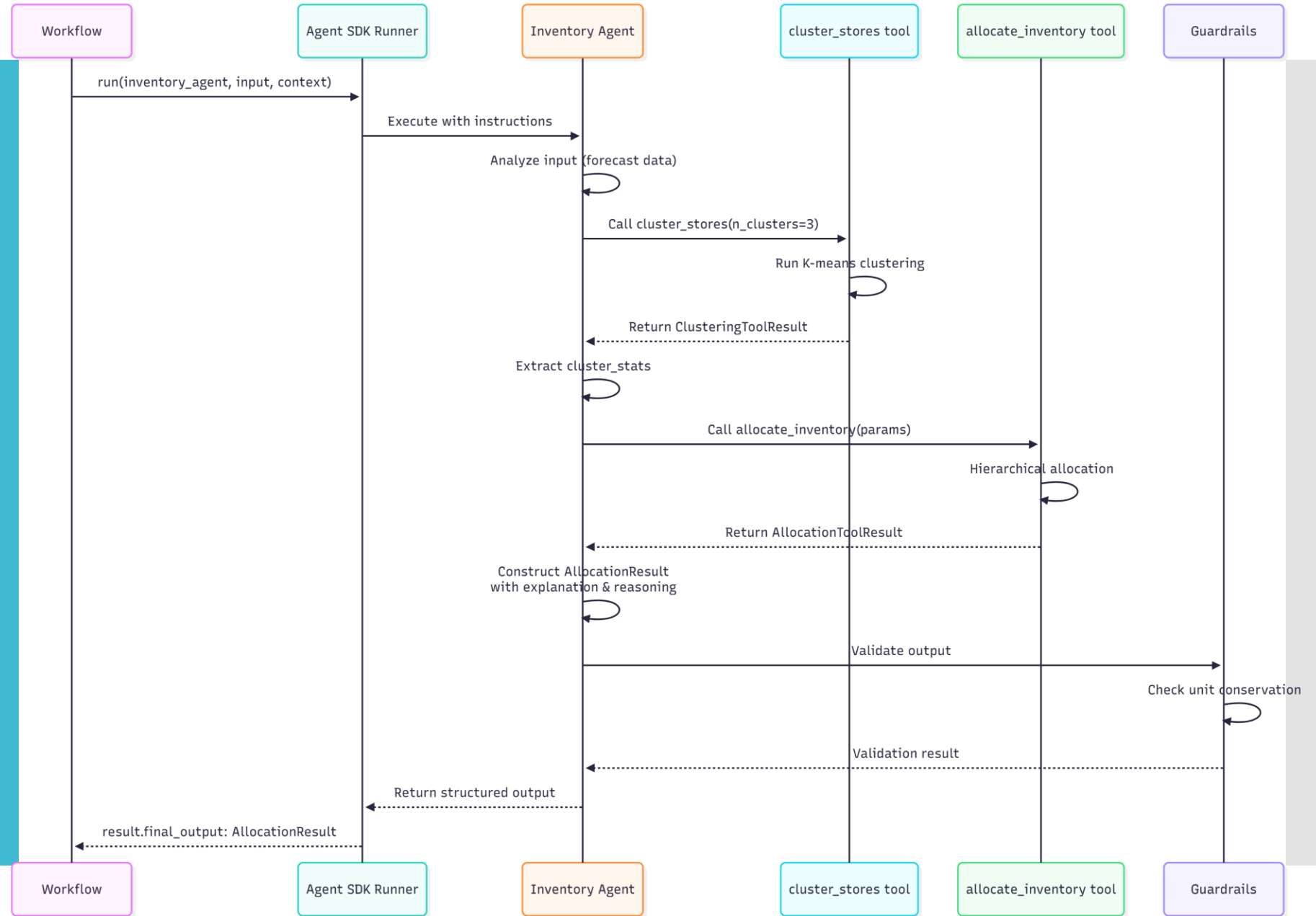


Inventory Agent: Allocation to Stores Cont.

Layer 3: Store allocation by
calculating individual
stores



Inventory Agent: Overall Execution Flow



Variance Agent

The Variance Agent determines whether a forecast needs to be updated by comparing actual sales against predictions. It's the decision gate that triggers the Reforecast Agent.

Variance Check

The agent first calls the `analyze_variance_data` tool to gather comprehensive metrics including weekly variance breakdowns and trend direction (comparing recent vs earlier variance magnitudes)

LLM Reasoning

Then the agent reasons over the data in the first step using a structured decision framework—assessing severity, hypothesizing likely causes (promotion effects, stockouts, competition, seasonality shifts, data quality), and evaluating whether variance is improving, stable, or worsening over time

Recommendation Output

It produces a structured `VarianceAnalysis` output containing the key decision (`should_reforecast: true/false`) and specific reasoning for that recommendation.

Reforecast Agent

The Reforecast Agent is triggered by Variance Agent's reforecast recommendation. If a "Reforecast" is recommended, the Reforecast Agent will use its Bayesian Reforecast tool to update the forecast curve.

Prior Distribution

The agent will read the previous forecast as a prior belief. This will be updated with new evidence i.e. observed actual sales data.

New Evidence

For each week with actuals, it calculates residuals (actual minus forecast) to determine the observed bias and its variance.

Statistical Significance

It then computes a t-statistic to assess statistical significance—with weak evidence ($t < 1$) applying only 30% of the bias, moderate evidence ($t 1-2$) applying 60%, and strong evidence ($t > 2$) applying 90%.

Forecast Curve Update

For remaining weeks, the adjustment is applied additively to the prior forecast mean, but decays exponentially (at rate 0.1 per week).

[Updated Forecast = prior + observed_bias * significance_factor * decay]

Reallocation Agent

The Reallocation agent will first gauge the necessity of reallocation and if a Reallocation is recommended, it will render the reallocation strategy based on user's strategy preference.

Velocity and Categorization

The agent will first calculate a velocity index for each store—defined as actual sales rate divided by expected sales rate based on linear sell-through assumptions—then categorizing stores as high performers (velocity > 1.15), underperformers (velocity < 0.85), or on-target.

Reallocation Urgency

For high performers, it calculates weeks of supply (remaining inventory divided by weekly sales rate) to assess stockout urgency and assigns transfer priority accordingly (high if WOS < 50% of remaining season, medium if velocity > 1.3 or WOS < 75%, low otherwise).

DC-to-store

DC-to-store releases allocate distributable DC inventory proportionally to each high performer's velocity excess above 1.0, sorted by urgency.

Store&DC-to-store

Store&DC-to-store transfers move 30% of underperformers' remaining inventory to high performers

Pricing Agent – Architecture

- Four-layer design
- Clear separation of concerns
- OpenAI Agent SDK

The Pricing Agent uses a **four-layer architecture** built on the OpenAI Agents SDK, ensuring modularity and maintainability. Each layer has distinct responsibilities, enabling independent testing and updates.

Workflow Layer

Orchestrates execution flow, determines when to trigger markdown checks based on week number and sell-through thresholds

`pricing_workflow.py`

Agent Layer

Makes intelligent decisions using LLM capabilities, interprets data and generates business explanations

`pricing_agent.py`

Tool Layer

Executes the core markdown calculation using $\text{Gap} \times \text{Elasticity}$ formula with configurable parameters

`pricing_tools.py`

Validation Layer

Applies guardrails to ensure outputs meet business rules (40% cap, 5% rounding, valid ranges)

`pricing_guardrails.py`

Pricing Agent – Core Components

- Agent definition
- Calculation tool
- Structured output

- Pricing Agent Definition

The agent is configured with specific instructions, tools, and output types. It uses the LLM to interpret sales data and decide on markdown strategies.

```
pricing_agent = Agent(  
    name="Pricing Agent",  
    instructions="Calculate markdown using Gap × Elasticity...",  
    tools=[calculate_markdown],  
    output_type=MarkdownResult,  
    output_guardrails=[validate_markdown_output])
```

- Calculate Markdown Tool

The core computation function that applies the pricing formula with business constraints..

```
@function_tool  
def calculate_markdown(current_sell_through, target=0.6, elasticity=2.0):  
    # Gap = Target - Current  
    # Raw = Gap × Elasticity  
    # Final = min(Raw, 40%) rounded to 5%  
    return MarkdownToolResult(...)
```

- Markdown Result Schema

`current_sell_through` - Current sales rate (e.g., 45%)

`target_sell_through` - Target rate (default 60%)

`gap` - Difference to close

`explanation` - Business justification

`final_markdown` - Recommended discount

Pricing Agent – Season Workflow Integration

- Phase 4 of 5
- Depends on forecast & allocation
- Feeds into variance analysis

The Pricing Agent operates as **Phase 4** in the complete Season Workflow orchestrated by `season_workflow.run_full_season()`. It receives computed sell-through data from earlier phases and generates markdown recommendations that feed into the final variance analysis.



Input Dependencies

From Phase 1: Weekly demand forecasts per store
From Phase 2: Initial inventory allocation across stores
From Phase 3: Reallocated inventory + actual sales data

Sell-Through Calculation

$$\text{current_sell_through} = \frac{\text{actual_sales}}{\text{allocated_inventory}}$$

This ratio determines whether markdown intervention is needed to meet seasonal targets.

Trigger Conditions

✓ **Week Check:** `week_number` ≥ `markdown_week` (default: 6)
✓ **Performance Check:** `sell_through` < `markdown_threshold` (default: 60%)
Both conditions must be met to trigger execution

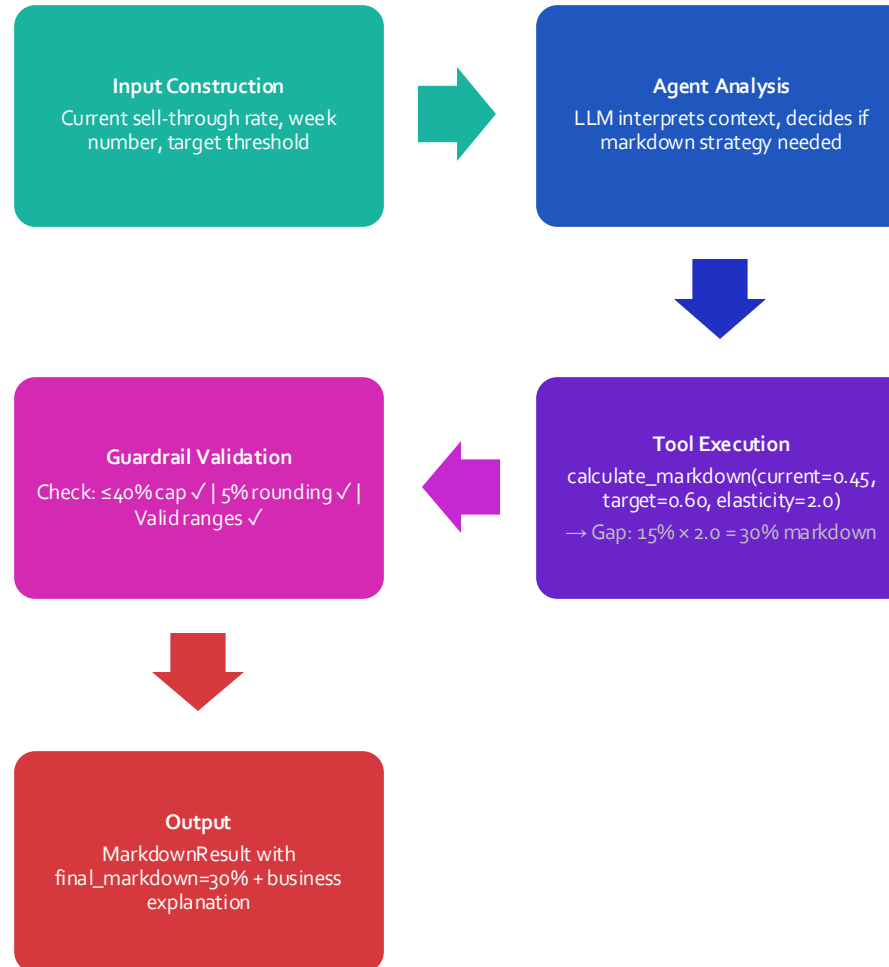
Output to Phase 5

- **MarkdownResult** object stored in `SeasonResult.markdown`
- Variance Agent uses markdown data to analyze sales deviation
- Results displayed in Streamlit UI via `render_pricing_section()`

Pricing Agent – Internal Workflow

- Input processing
- Tool invocation
- Output validation

When triggered, the Pricing Agent follows a structured workflow to analyze sales performance and generate markdown recommendations. The workflow is orchestrated by `run_markdown_check()` function, which constructs input prompts, invokes the agent via `Runner.run()`, and returns validated results.



Key Functions

- `should_run_markdown_check()` - Determines if checkpoint week is reached
- `run_markdown_check()` - Core execution function with UI hooks support
- `run_markdown_if_needed()` - Conditional wrapper checking thresholds

Calculation Example

- Current: 45% | Target: 60% | Gap: 15%
- Raw: $15\% \times 2.0 = 30\%$
- **Final: 30% (within cap, rounded)**

UI Integration

- Supports phase_start and phase_end hooks for real-time status updates in Streamlit dashboard

Pricing Agent – Result & Validation

- Structured output
- Hard & Soft rules
- Business explanation

Example output

```
{  
  "current_sell_through": 0.45,  
  "target_sell_through": 0.60,  
  "gap": 0.15,  
  "elasticity": 2.0,  
  "raw_markdown": 0.30,  
  "final_markdown": 0.30,  
  "explanation": "30% markdown recommended..."  
}
```

Hard Validation

- $\text{Markdown} \leq 40\%$ (profit protection)
- $\text{Markdown} \geq 0\%$ (no negative values)
- Sell-through in 0-100% range
- Elasticity > 0

Failures block execution

Soft Validation

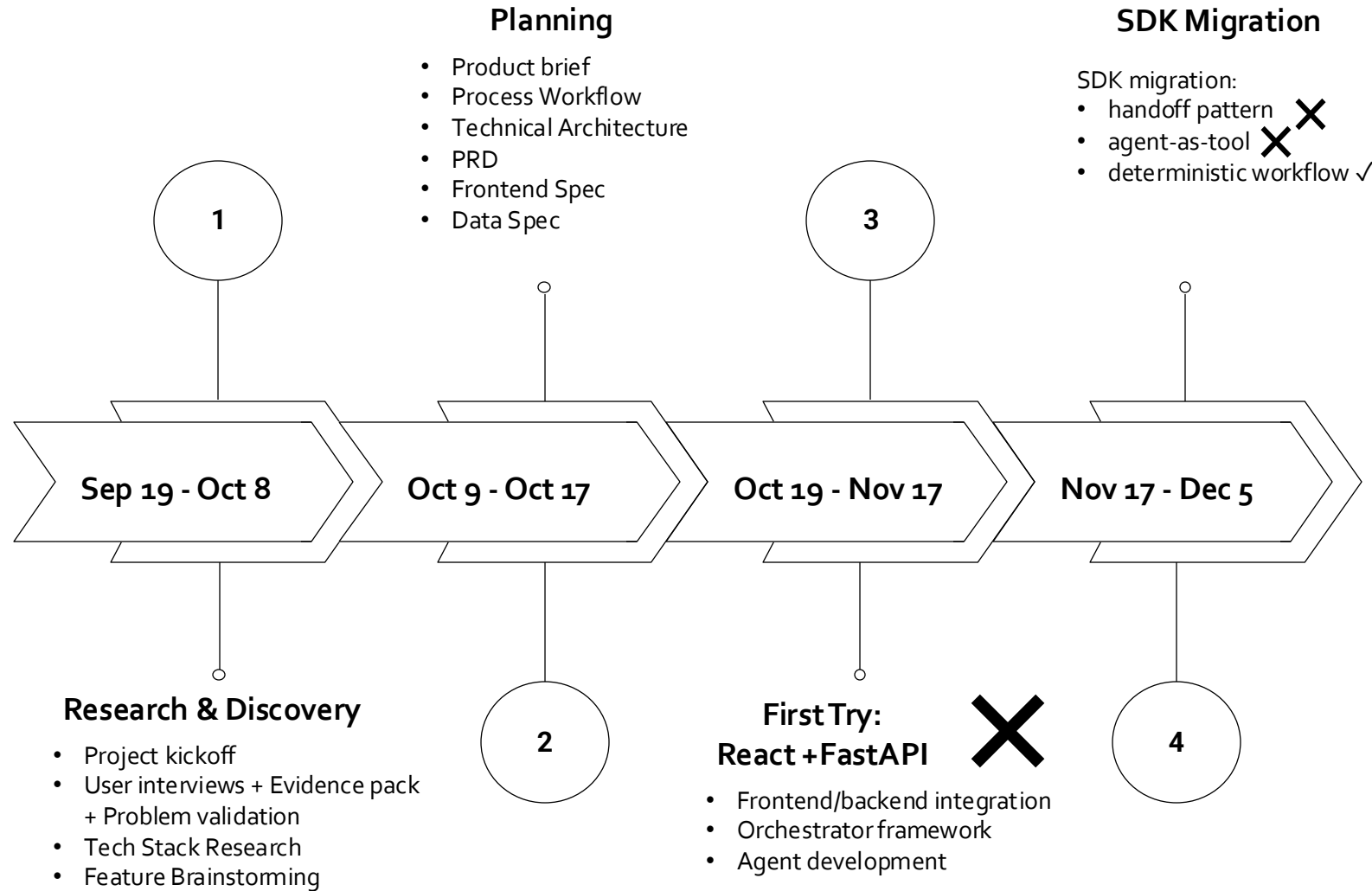
- Early season high markdown
- $\text{Markdown} > 35\%$ warning
- Low elasticity (< 1.0) alert
- Very low sell-through ($< 20\%$)

Generates warnings only

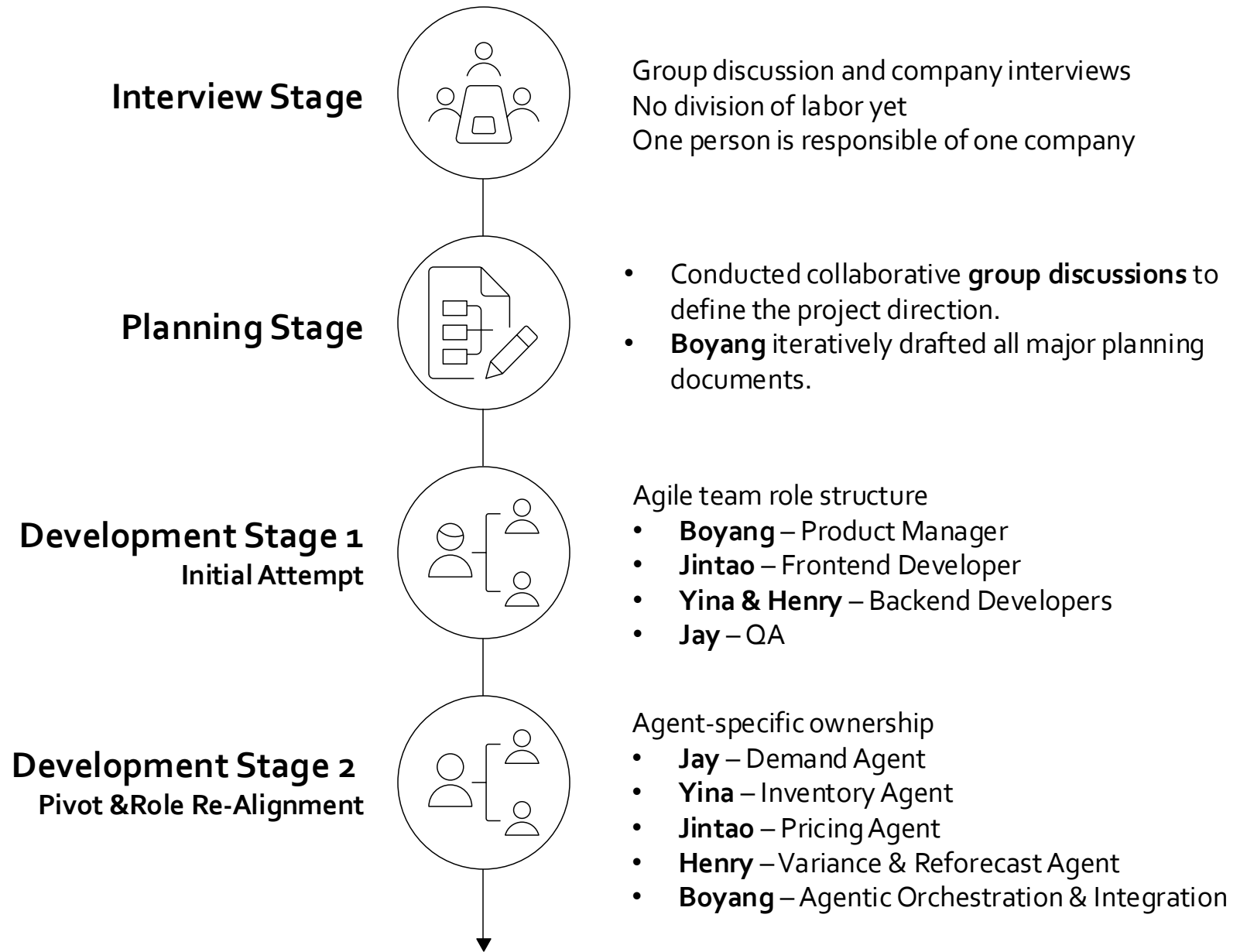
Business Value

The agent provides actionable markdown recommendations with clear explanations, enabling merchandising teams to make data-driven pricing decisions while respecting profit margin constraints.

Development Journey



Team Collaboration



Reflections

Our Missteps

- Difficulty Applying Agile in Practice

We tried to follow an Agile development lifecycle but struggled to sprint quickly when requirements changed. Our process wasn't flexible enough to adjust mid-development.

- Misaligned Development Priorities

We focused too early on individual agent features instead of building the agentic framework first, which caused unnecessary rework.

- Overly Complex Architecture Choice

Choosing a FastAPI + React architecture was overly ambitious. It shifted attention away from exploring AI agents toward solving dev challenges we weren't aiming to practice.

- Insufficient research of the Agent SDK at first

We didn't research the OpenAI Agents SDK deeply enough at first. Missing understanding of its patterns led to repeated redesigns and inefficiencies

Reflections

What we learned?

- Industry Understanding: By interviews, we learned how retail industry workflow and their pain points
- PoC end to end experience: tried software dev lifecycle from requirement gathering -> ideation -> planning -> development -> pivoting -> have a functional demo
- Tried out spec-drive AI coding, applied into practice with structuralized documentations
- AI agentic framework, better understand what it can do, and the potential, hand on practice on building agent and tools

PoC -> Production

- Expand Agent Tooling: we should use or build more tools for each agent to use. eg: we could use prebuild web search tool for the variance and reforecast agents to make decision on whether we should trigger reforecast
- Sessions
- Evals
- MCPs
- Temporal
- Reasoning llm models
- Streamlit -> react + FastAPI, cloud deployment
- Data pipeline, right now we are just uploading cleaned csv files, which is mock, for production we need to build data pipeline before that